coursera

Build a Complete MCP Server with Security and Monitoring

Objective

The goal isn't just to build an MCP server—it’s to design and implement a secure, resilient, and high-performance integration layer that meets real-world production demands. Are you balancing strict security, auditability, and role-based control with scalability and performance? Are you delivering an architecture that can handle thousands of daily interactions while ensuring reliability, observability, and maintainability in a live environment?

Scenario

You're the lead architect at DataFlow Solutions, a consulting company that helps enterprises integrate AI systems with their existing data infrastructure. Your latest client is a mid-sized e-commerce company that wants to deploy an AI-powered customer service assistant.

The assistant needs to access customer order history, product information, and support ticket data to provide personalized help. However, the company has strict security requirements—customer data must be protected, access must be audited, and the system must be resilient to failures.

Your task is to build a production-ready MCP server that provides secure, monitored access to customer service data. This server will be deployed in a production environment with thousands of daily interactions, so it must be robust, secure, and performant.

coursera

The company's technical requirements include

- API key-based authentication with role-based access control
- Comprehensive audit logging of all data access
- Health monitoring and metrics collection
- Graceful error handling and recovery
- Performance optimization for high-traffic scenarios

Instructions for learners

Build a complete MCP server implementation that demonstrates secure, production-ready AI integration patterns with comprehensive monitoring and error handling.

Refine the learner's task with completion criteria

Your MCP server implementation must include the following components

coursera

1. Server Architecture and Setup

- Implement a well-structured MCP server class with proper initialization
- Include connection pooling and resource management
- Set up proper logging and configuration management
- Provide clear documentation on server setup and deployment

coursera

2. Authentication and Authorization

- Implement API key-based authentication
- Create role-based access control for different user types
- Include permission checking for all resource and tool access
- Provide secure error handling that doesn't leak sensitive information

coursera

3. Resource Implementation

- Define at least 3 different resource types (e.g., customers, orders, support tickets)
- Implement proper URI patterns and schema definitions
- Include data filtering based on user permissions
- Add caching for frequently accessed resources

coursera

4. Tool Implementation

- Implement at least 2 tools that modify data (e.g., create ticket, update order)
- Include input validation and sanitization
- Provide proper error handling and rollback mechanisms
- Add audit logging for all tool executions

coursera

5. Security Features

- Implement TLS/SSL support for encrypted communications
- Add rate limiting to prevent abuse
- Include comprehensive audit logging
- Provide secure configuration management

coursera

6. Monitoring and Health Checks

- Implement health check endpoints
- Add metrics collection for performance monitoring
- Include error tracking and alerting
- Provide operational dashboards or monitoring interfaces

coursera

7. Error Handling and Resilience

- Implement graceful degradation when dependencies fail
- Add circuit breaker patterns for external service calls
- Include proper timeout handling
- Provide comprehensive error logging and recovery

coursera

8. Performance Optimization

- Implement connection pooling for database access
- Add intelligent caching strategies
- Include asynchronous processing for long-running operations
  - --Optimize for high-concurrency scenarios

coursera

9. Documentation and Testing

- Provide comprehensive API documentation
- Include unit tests for core functionality
- Add integration tests for end-to-end workflows
- Provide deployment and configuration guides

coursera

10. Production Readiness

- Include Docker containerization
- Provide production deployment configurations
- Add monitoring and alerting setup
- Include disaster recovery and backup procedures

coursera

Technical Requirements

- Use Python, Node.js, or another language appropriate for MCP development
- Follow MCP protocol specifications accurately
- Include proper error handling and logging throughout
- Demonstrate understanding of security best practices
- Show consideration for production deployment requirements

Submission

Submission Guidelines

Submit your complete MCP server implementation as a structured codebase including:

- Source code with proper organization and documentation
- Configuration files and deployment scripts
- Test suite demonstrating functionality
- README with setup and deployment instructions
- Architecture documentation explaining design decisions

Reflection

Engage in reflection Transition to Dialogue

After completing your MCP server implementation, you'll participate in a comprehensive review process where you'll defend your design decisions, explain your security approach, and demonstrate how your server handles various scenarios. This dialogue will help you refine your implementation and ensure it meets production standards.